

MODUL PRAKTIKUM STRUKTUR DATA

IMPLEMENTASI ALGORITMA SEDERHANA DAN ANALISIS LANGKAH

1. JUDUL PRAKTIKUM

Implementasi Algoritma Sederhana dan Analisis Langkah

2. IDENTITAS PRAKTIKUM

Komponen	Deskripsi
Mata Kuliah	Struktur Data
Kode/SKS	SD123 / 1 SKS Praktikum
Pertemuan ke-	1 (Satu)
Waktu Pelaksanaan	170 menit

3. CAPAIAN PEMBELAJARAN PRAKTIKUM

CPMK Terkait

Kode	Capaian Pembelajaran Mata Kuliah
CPMK-1	Mahasiswa mampu mengimplementasikan algoritma dasar menggunakan bahasa pemrograman Python
CPMK-2	Mahasiswa mampu menganalisis langkah-langkah algoritma dan menentukan kompleksitas waktu sederhana

Kode	Capaian Pembelajaran Mata Kuliah
CPMK-3	Mahasiswa mampu mendokumentasikan dan mempresentasikan hasil analisis algoritma

Sub-CPMK

Kode	Sub-Capaian Pembelajaran
Sub-CPMK-1	Menerjemahkan algoritma sederhana ke dalam kode program Python dengan benar
Sub-CPMK-2	Melakukan tracing (penelusuran) manual terhadap eksekusi algoritma
Sub-CPMK-3	Menghitung jumlah operasi dasar dalam algoritma sebagai fungsi dari ukuran input
Sub-CPMK-4	Menganalisis pengaruh ukuran input terhadap waktu eksekusi secara empiris
Sub-CPMK-5	Menyusun laporan praktikum yang sistematis sesuai format akademik

4. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

No	Tujuan	Indikator Ketercapaian
1	Mengimplementasikan algoritma pencarian linear, pencarian nilai maksimum, dan pengecekan bilangan prima dalam bahasa Python	Program berjalan tanpa error menghasilkan output sesuai spesifikasi
2	Melakukan tracing langkah demi langkah eksekusi algoritma untuk memahami alur kerja program	Mahasiswa mampu menjelaskan eksekusi untuk input tertentu
3	Menghitung jumlah operasi dasar yang dilakukan algoritma sebagai fungsi dari ukuran input	Tabel hasil penghitungan langkah untuk berbagai ukuran input
4	Menganalisis hubungan antara ukuran input dan waktu eksekusi secara empiris	Grafik waktu vs ukuran input menunjukkan pola yang sesuai dengan teori
5	Menyusun laporan praktikum yang terstruktur dengan analisis yang tepat	Laporan memenuhi semua komponen yang ditentukan

5. DASAR TEORI

5.1 Konsep Algoritma

Definisi Algoritma

Algoritma adalah urutan langkah-langkah logis yang sistematis untuk menyelesaikan suatu masalah. Dalam konteks pemrograman, algoritma menjadi fondasi dari setiap program yang kita buat. Sebuah algoritma yang baik harus memenuhi kriteria berikut yang dikemukakan oleh Donald Knuth dalam bukunya "The Art of Computer Programming":

1. **Input:** Memiliki 0 atau lebih input yang didefinisikan dengan jelas.
2. **Output:** Menghasilkan minimal 1 output yang relevan dengan masalah.
3. **Definiteness:** Setiap langkah didefinisikan secara tepat dan tidak ambigu.
4. **Finiteness:** Algoritma harus berakhir setelah sejumlah langkah terbatas.
5. **Effectiveness:** Setiap langkah dapat dilaksanakan dalam waktu yang masuk akal.

Representasi Algoritma

Algoritma dapat direpresentasikan dalam beberapa bentuk:

- **Pseudocode:** Deskripsi langkah-langkah menggunakan bahasa manusia yang terstruktur, mirip dengan kode program tetapi tidak terikat pada sintaks bahasa pemrograman tertentu.
- **Flowchart:** Diagram alir yang menggunakan simbol-simbol standar untuk menggambarkan alur logika.
- **Code program:** Implementasi langsung dalam bahasa pemrograman tertentu (dalam praktikum ini menggunakan Python).

5.2 Analisis Algoritma dan Operasi Dasar

Analisis algoritma bertujuan untuk mengukur efisiensi algoritma dalam hal waktu eksekusi dan penggunaan memori. Dalam praktikum ini, kita akan fokus pada analisis waktu dengan menghitung jumlah **operasi dasar** yang dilakukan.

Operasi dasar adalah operasi yang paling sering dilakukan dalam algoritma dan menjadi faktor utama penentu waktu eksekusi. Operasi dasar meliputi:

Jenis Operasi	Contoh	Simbol	Keterangan
Aritmatika	Penjumlahan, pengurangan, perkalian, pembagian	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	Operasi matematika dasar
Perbandingan	Lebih besar, lebih kecil, sama dengan	<code>></code> , <code><</code> , <code>==</code> , <code>!=</code> , <code>>=</code> , <code><=</code>	Digunakan dalam pengambilan keputusan

Jenis Operasi	Contoh	Simbol	Keterangan
Assignment	Pemberian nilai ke variabel	=	Menyimpan nilai ke memori
Logika	AND, OR, NOT	and, or, not	Operasi boolean
Input/Output	Membaca/menulis data	input(), print()	Interaksi dengan pengguna

Jumlah operasi dasar ini akan menjadi fungsi dari ukuran input (biasanya dinotasikan dengan n). Fungsi ini kemudian dinyatakan dalam notasi **Big-O** yang menggambarkan laju pertumbuhan waktu eksekusi.

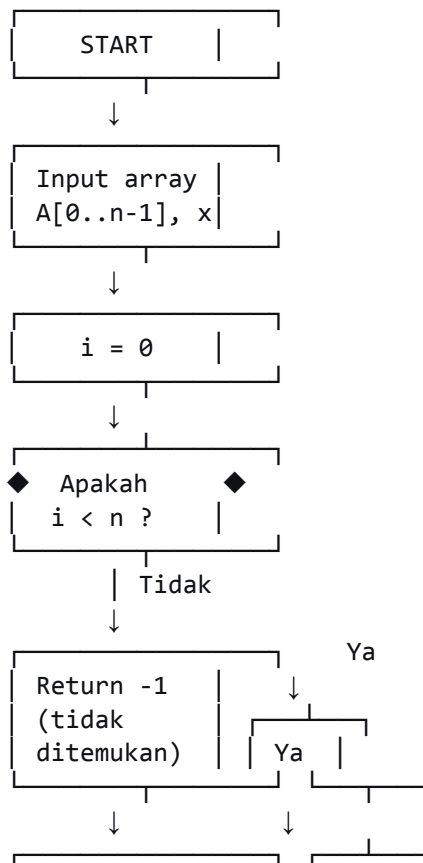
5.3 Algoritma 1: Pencarian Linear (Linear Search)

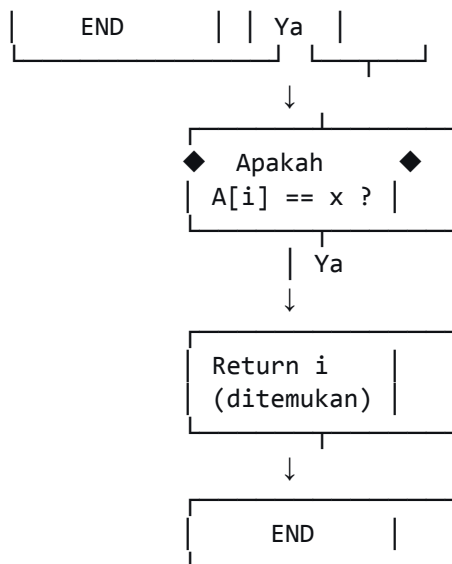
Deskripsi

Pencarian linear adalah algoritma paling sederhana untuk mencari nilai tertentu dalam sebuah larik (array). Algoritma ini bekerja dengan membandingkan nilai yang dicari dengan setiap elemen array secara berurutan dari indeks pertama hingga terakhir. Kompleksitas waktu algoritma ini adalah $O(n)$ karena dalam kasus terburuk, kita harus memeriksa semua elemen.

Flowchart Linear Search

text





Pseudocode Linear Search

text
 ALGORITMA LinearSearch
 INPUT: array A[0..n-1], nilai x
 OUTPUT: indeks i jika x ditemukan, -1 jika tidak

1. Untuk i dari 0 sampai n-1:
2. Jika A[i] == x maka:
3. Kembalikan i
4. Akhir Jika
5. Akhir Untuk
6. Kembalikan -1

Ilustrasi Tracing

text
 Array: [23, 45, 12, 67, 89, 34, 56, 78]
 Cari: 67

Langkah 1: i=0, A[0]=23 ≠ 67
 Langkah 2: i=1, A[1]=45 ≠ 67
 Langkah 3: i=2, A[2]=12 ≠ 67
 Langkah 4: i=3, A[3]=67 → ditemukan, kembalikan i=3
 Jumlah langkah = 4 perbandingan

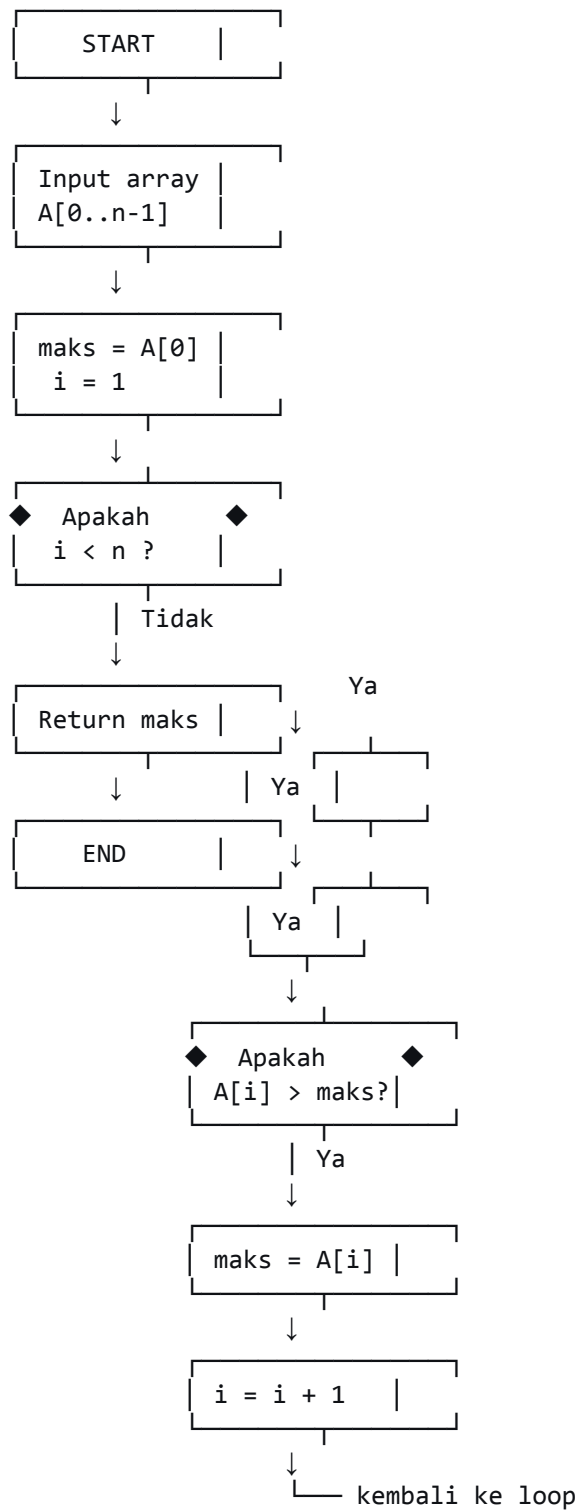
5.4 Algoritma 2: Mencari Nilai Maksimum

Deskripsi

Algoritma untuk menemukan nilai terbesar dalam sebuah larik. Algoritma ini mengasumsikan elemen pertama sebagai nilai maksimum sementara, lalu membandingkan dengan elemen-elemen berikutnya. Jika ditemukan nilai yang lebih besar, nilai maksimum diperbarui. Kompleksitas waktu algoritma ini juga $O(n)$ karena melakukan $n-1$ perbandingan.

Flowchart Mencari Maksimum

text



Pseudocode Mencari Maksimum

text

ALGORITMA CariMaksimum

INPUT: array $A[0..n-1]$ ($n > 0$)

OUTPUT: nilai maksimum dalam A

1. $\text{maks} = A[0]$

2. Untuk i dari 1 sampai $n-1$:
3. Jika $A[i] > \text{maks}$ maka:
4. $\text{maks} = A[i]$
5. Akhir Jika
6. Akhir Untuk
7. Kembalikan maks

Ilustrasi Tracing

text

Array: [23, 45, 12, 67, 89, 34, 56, 78]

Langkah 1: $\text{maks} = 23, i=1$

Langkah 2: $A[1]=45 > 23 \rightarrow \text{maks} = 45$ (assignment)

Langkah 3: $A[2]=12 \leq 45 \rightarrow \text{maks}$ tetap 45

Langkah 4: $A[3]=67 > 45 \rightarrow \text{maks} = 67$ (assignment)

Langkah 5: $A[4]=89 > 67 \rightarrow \text{maks} = 89$ (assignment)

Langkah 6: $A[5]=34 \leq 89 \rightarrow \text{maks}$ tetap

Langkah 7: $A[6]=56 \leq 89 \rightarrow \text{maks}$ tetap

Langkah 8: $A[7]=78 \leq 89 \rightarrow \text{maks}$ tetap

Hasil: 89

Jumlah perbandingan: 7

Jumlah assignment: 4 (termasuk assignment awal)

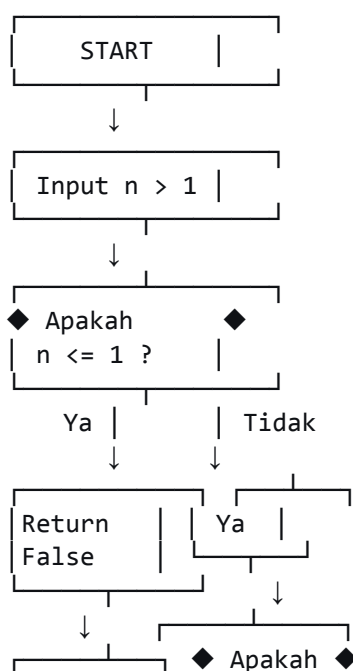
5.5 Algoritma 3: Pengecekan Bilangan Prima

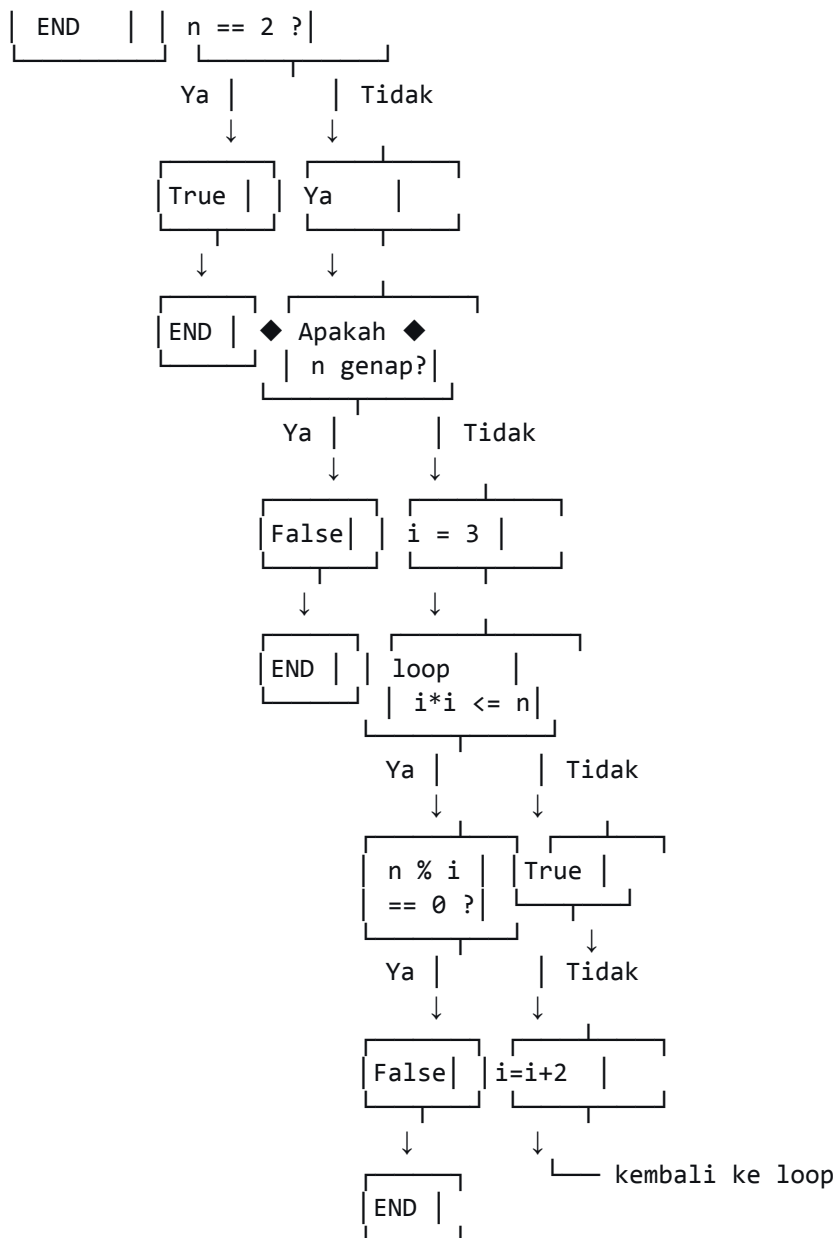
Deskripsi

Bilangan prima adalah bilangan bulat positif yang hanya memiliki dua faktor, yaitu 1 dan bilangan itu sendiri. Algoritma yang efisien untuk mengecek bilangan prima hanya perlu memeriksa pembagi hingga akar kuadrat bilangan tersebut, dan hanya bilangan ganjil setelah mengecualikan 2. Kompleksitas waktu algoritma ini adalah $O(\sqrt{n})$.

Flowchart Cek Prima (Optimasi)

text





Pseudocode Cek Prima

text

ALGORITMA CekPrima

INPUT: bilangan bulat $n > 1$

OUTPUT: True jika prima, False jika tidak

1. Jika $n \leq 1$: Kembalikan False
2. Jika $n == 2$: Kembalikan True
3. Jika $n \% 2 == 0$: Kembalikan False
4. $i = 3$
5. Selama $i * i \leq n$:
 6. Jika $n \% i == 0$:
 7. Kembalikan False
 8. $i = i + 2$
9. Akhir Selama
10. Kembalikan True

Ilustrasi Tracing

text

Cek 97:

$n=97 > 1$, $n \neq 2$, n ganjil, $i=3$

Iterasi 1: $3^2=9 \leq 97$, $97\%3=1 \neq 0 \rightarrow i=5$

Iterasi 2: $5^2=25 \leq 97$, $97\%5=2 \neq 0 \rightarrow i=7$

Iterasi 3: $7^2=49 \leq 97$, $97\%7=6 \neq 0 \rightarrow i=9$

Iterasi 4: $9^2=81 \leq 97$, $97\%9=7 \neq 0 \rightarrow i=11$

Iterasi 5: $11^2=121 > 97 \rightarrow \text{stop}$

Hasil: True (prima)

Jumlah iterasi: 4 (karena iterasi ke-5 hanya pengecekan kondisi)

5.6 Analisis Jumlah Langkah dan Kompleksitas

Konsep Analisis Langkah

Analisis langkah bertujuan untuk menghitung jumlah operasi dasar yang dilakukan algoritma sebagai fungsi dari ukuran input (n). Hasil analisis ini dinyatakan dalam notasi **Big-O** yang menggambarkan laju pertumbuhan waktu eksekusi.

Tabel Kompleksitas untuk Ketiga Algoritma

Algoritma	Best Case	Worst Case	Average Case	Notasi Big-O
Linear Search	1 langkah (elemen pertama)	n langkah (elemen terakhir/tidak ada)	$n/2$ langkah	$O(n)$
Cari Maksimum	$n-1$ perbandingan + 1 assignment	$n-1$ perbandingan + $n-1$ assignment	$n-1$ perbandingan + $-\log n$ assignment	$O(n)$
Cek Prima (optimasi)	~ 1 langkah (genap/kecil)	$\sim \sqrt{n}/2$ langkah	$\sim \sqrt{n}/2$ langkah	$O(\sqrt{n})$

6. ALAT DAN BAHAN

6.1 Perangkat Keras

No	Perangkat	Spesifikasi Minimal	Keterangan
1	Komputer/Laptop	Prosesor 1.5 GHz, RAM 2 GB, HDD 20 GB free	Untuk menulis dan menjalankan kode

No	Perangkat	Spesifikasi Minimal	Keterangan
2	Mouse	-	Untuk navigasi
3	Keyboard	-	Untuk mengetik
4	Media penyimpanan	Flashdisk 4 GB atau akun cloud	Untuk menyimpan file

6.2 Perangkat Lunak

No	Perangkat Lunak	Versi	Fungsi	Sumber
1	Python	3.8 atau lebih baru	Interpreter untuk menjalankan kode	python.org
2	Visual Studio Code	Terbaru	Editor kode (alternatif: PyCharm, IDLE, Sublime)	code.visualstudio.com
3	Web Browser	Terbaru	Untuk mengakses dokumentasi	-
4	Spreadsheet (Excel/LibreOffice)	-	Untuk membuat grafik (opsional)	-
5	Git (opsional)	-	Version control	git-scm.com

6.3 Dataset/Bahan Pendukung

No	Bahan	Deskripsi
1	Template laporan	Disediakan oleh asisten praktikum (format .docx)
2	Modul praktikum	Dokumen ini (format .pdf)
3	Dataset contoh	Akan dibuat sendiri oleh mahasiswa

7. KESELAMATAN KERJA

Meskipun praktikum ini tidak menggunakan peralatan berbahaya, tetaplah memperhatikan hal-hal berikut untuk kenyamanan dan keamanan bersama:

No	Aspek	Panduan Keselamatan
1	Kelistrikan	Pastikan kabel daya terhubung dengan baik, hindari kabel berantakan yang menyebabkan tersandung
2	Postur tubuh	Duduk dengan posisi tegak, layar sejajar mata, jarak pandang 40–50 cm untuk menghindari kelelahan
3	Keselamatan data	Simpan kode program secara berkala (Ctrl+S), backup ke cloud atau flashdisk setiap 30 menit
4	Istirahat	Lakukan peregangan setiap 30–45 menit untuk mengurangi ketegangan mata dan otot
5	Kebersihan	Jaga kebersihan area kerja, tidak makan/minum di dekat perangkat elektronik
6	Etika laboratorium	Tidak membuat keributan, menjaga ketenangan, dan menggunakan perangkat sesuai kebutuhan

Mulai praktikum

8. LANGKAH KERJA PRAKTIKUM

8.1 Tahap Persiapan (20 menit)

Langkah	Instruksi	Keterangan
1	Nyalakan komputer dan login ke sistem operasi	Pastikan semua perangkat berfungsi normal
2	Buka terminal/command prompt, ketik <code>python --version</code>	Verifikasi instalasi Python. Harus muncul <code>3.x.x</code>
3	Buat folder kerja dengan nama <code>Praktikum1_NIM_Nama</code>	Contoh: <code>Praktikum1_12345678_Ahmad</code>
4	Buka text editor/IDE (VSCode, PyCharm, dll)	Siapkan untuk menulis kode
5	Pelajari kembali dasar teori di modul ini	Pahami algoritma yang akan diimplementasikan

8.2 Tahap Implementasi Linear Search (25 menit)

Langkah	Instruksi	Kode/Perintah
1	Buat file baru dengan nama <code>linear_search.py</code>	Di dalam folder kerja
2	Tulis fungsi <code>linear_search(arr, x)</code>	<pre>python def linear_search(arr, x): """ Fungsi untuk mencari indeks elemen x dalam array arr Mengembalikan indeks ditemukan, -1 jika tidak """ for i in range(len(arr)): if arr[i] == x: return i return -1</pre>
3	Tulis fungsi <code>main()</code> untuk pengujian	<pre>python def main(): data = [23, 45, 12, 67, 89, 34, 56, 78] cari = 67 hasil = linear_search(data, cari) if hasil != -1: print(f"{cari} ditemukan di indeks {hasil}") else: print(f"{cari} tidak ditemukan")</pre>
4	Tambahkan pemanggil <code>main()</code>	<pre>python if __name__ == "__main__": main()</pre>
5	Jalankan program	<code>python linear_search.py</code> di terminal
6	Amati output	Catat hasilnya

8.3 Tahap Penambahan Penghitung Langkah (25 menit)

8.4 Tahap Implementasi Pencarian Maksimum (25 menit)

Langkah	Instruksi	Kode/Perintah
1	Duplikasi file <code>linear_search.py</code> menjadi <code>linear_search_count.py</code>	Copy-paste
2	Modifikasi fungsi <code>linear_search</code> dengan counter	<pre>python def linear_search_count(arr, x): count = 0 for i in range(len(arr)): count += 1 if arr[i] == x: return i, count return -1, count</pre>
3	Modifikasi <code>main()</code> untuk menguji berbagai kasus	<pre>python def main(): data = [23, 45, 12, 67, 89, 34, 56, 78] test_values = [67, 23, 78, 99] for val in test_values: idx, steps = linear_search_count(data, val) if idx != -1: print(f"Cari {val}: ditemukan di indeks {idx}, langkah: {steps}") else: print(f"Cari {val}: tidak ditemukan, langkah: {steps}")</pre>
4	Jalankan program dan catat hasil	Isi Tabel 1.1 di lembar pengamatan
5	Analisis mengapa jumlah langkah berbeda	Diskusikan dengan teman

Langkah	Instruksi	Kode/Perintah
1	Buat file baru <code>max_search.py</code>	-
2	Tulis fungsi <code>find_max</code> dengan penghitung	<pre>python def find_max(arr): if not arr: return None, 0, 0 max_val = arr[0] comp_count = 0 assign_count = 1 for i range(1, len(arr)): comp_count += 1 if arr[i] > max_val: max_val = arr[i] assign_count += 1 return max_val, comp_count, assign_count</pre>
3	Buat data uji dengan karakteristik berbeda	<pre>python data_acak = [23, 45, 12, 67, 89, 34, 56, 78] data_menurun = [100, 90, 80, 70, 60] data_menaik = [10, 30, 40, 50]</pre>
4	Tulis <code>main()</code> untuk menguji ketiga data	<pre>python datasets = [("Acak", data_acak), ("Menurun", data_menurun), ("Menaik", data_menaik)] for nama, dat datasets: maks, comp, assign = find_max(data) print(f" {nama}: {data}") print(f"Maks: {maks}, Perbandingan: { Assignment: {assign}") print(f"Total langkah: {comp + assign}")</pre>
5	Jalankan program dan catat hasil	Isi Tabel 1.2
6	Analisis perbedaan assignment	Mengapa data menaik menghasilkan assignment lebih banyak?

8.5 Tahap Implementasi Cek Bilangan Prima (25 menit)

Langkah	Instruksi	Kode/Perintah
1	Buat file baru <code>prime_check.py</code>	-
2	Tulis fungsi <code>is_prime</code> dengan penghitung	<pre>python import math def is_prime(n): if n <= 1: return False 0 if n == 2: return True, 0 if n % 2 == 0: return False, 1 iter_count = 0 i = 3 while i * i <= n: iter_count += 1 if i == 0: return False, iter_count i += 2 return True, iter_count</pre>
3	Buat daftar bilangan uji	<pre>python test_numbers = [2, 3, 17, 19, 23, 97, 100, 101, 997, 999, 1000]</pre>
4	Tulis <code>main()</code> untuk menampilkan hasil	<pre>python print("="*60) print("PENGUJIAN BILANGAN PRIMA") print("="*60) print(f"{'n':<10} {'Prima?':<8} {'Iterasi':<8} {'√n':<10}") print("-"*60) for n in test_numbers: prime = is_prime(n) print(f"{n:10} {prime:8} {iter_count:8} {math.sqrt(n):10}")</pre>

Langkah	Instruksi	Kode/Perintah
		<pre>iterasi = is_prime(n) akar = int(math.sqrt(n)) status = if prima else "Tidak" print(f"{n:<10} {status:<8} {iterasi:<10} {akar:<10}")</pre>
5	Jalankan program dan catat hasil	Isi Tabel 1.3
6	Analisis hubungan iterasi dengan $\sqrt{n}$	Apakah iterasi selalu $\leq \sqrt{n}/2$?

8.6 Tahap Eksperimen Ukuran Data (25 menit)

Langkah	Instruksi	Kode/Perintah
1	Buat file baru <code>experiment.py</code>	-
2	Import modul yang diperlukan	<pre>python import time import random</pre>
3	Tulis fungsi linear search (tanpa counter)	<pre>python def linear_search(arr, x): for i in range(len(arr)): arr[i] == x: return i return -1</pre>
4	Tulis fungsi untuk generate data acak	<pre>python def generate_data(n): return [random.randint(1, 1000000) _ in range(n)]</pre>
5	Tulis fungsi pengukur waktu	<pre>python def measure_time(search_func, arr, x): start = time.perf_counter() search_func(arr, x) end = time.perf_counter() return end - start</pre>
6	Tentukan ukuran data yang akan diuji	<pre>python sizes = [1000, 5000, 10000, 50000, 100000]</pre>
7	Tulis <code>main()</code> untuk eksperimen	<pre>python def main(): print("="*60) print("EKSPERIMEN WAKTU L SEARCH") print("="*60) print(f"'Ukuran (n)':<15} {'Waktu (detik)':<20}") print("-"*60) for n in sizes: data = generate_data(n) target = data[-1] # cari elemen terakhir measure_time(linear_search, data, target) print(f"{n:<15} {waktu:<20.6f}") if __name__ == "__main__": main()</pre>
8	Jalankan program dan catat hasil	Isi Tabel 1.4
9	Analisis pola pertumbuhan waktu	Apakah waktu naik secara linear?

8.7 Tahap Dokumentasi dan Pengumpulan (25 menit)

Langkah	Instruksi
1	Rapikan semua file kode program
2	Isi semua tabel pengamatan dengan data yang diperoleh
3	Jawab pertanyaan diskusi secara singkat
4	Buat kesimpulan awal dari praktikum
5	Kumpulkan semua file dalam folder <code>Praktikum1_NIM_Nama</code>
6	Kompres folder menjadi file .zip
7	Upload ke platform e-learning yang ditentukan

9. TUGAS PRAKTIKUM

9.1 Soal Latihan Bertahap

Soal 1: Modifikasi Linear Search untuk Data Duplikat (Skor 20)

Buatlah fungsi `linear_search_all` yang mencari semua kemunculan nilai `x` dalam array. Fungsi harus mengembalikan:

- List berisi indeks-indeks tempat `x` ditemukan
- Jumlah langkah perbandingan yang dilakukan

Contoh:

```
python
data = [23, 45, 12, 67, 23, 34, 23, 78]
cari = 23
# Output: indeks [0, 4, 6], Langkah 8
```

Soal 2: Analisis Best dan Worst Case Pencarian Maksimum (Skor 20)

Buatlah data uji untuk algoritma pencarian maksimum yang menghasilkan:

- **Best case:** jumlah assignment minimum (hanya 1 kali assignment awal)
- **Worst case:** jumlah assignment maksimum (assignment terjadi di setiap iterasi, total n assignment)

Tunjukkan:

1. Kode untuk membuat data tersebut
2. Hitung jumlah perbandingan dan assignment
3. Jelaskan mengapa data tersebut menghasilkan best/worst case

Soal 3: Perbandingan Algoritma Prima Naif vs Optimasi (Skor 25)

Buatlah dua versi fungsi pengecekan prima:

1. **Versi naif:** Loop dari 2 sampai $n-1$
2. **Versi optimasi:** Loop sampai $\sqrt{n}$, hanya ganjil (seperti di modul)

Uji kedua fungsi untuk bilangan:

- 997 (prima besar)
- 1000 (bukan prima)
- 1000003 (prima besar)
- 10000000 (10 juta, bukan prima, genap)

Buat tabel perbandingan yang mencakup:

- Waktu eksekusi (gunakan `time.perf_counter()`)
- Jumlah iterasi
- Kesimpulan: berapa kali lebih cepat versi optimasi?

Soal 4: Grafik Pertumbuhan Waktu (Skor 15)

Gunakan data dari eksperimen ukuran data (Tabel 1.4) untuk membuat grafik menggunakan spreadsheet (Excel/LibreOffice) atau matplotlib (Python). Sumbu X adalah ukuran data, sumbu Y adalah waktu eksekusi. Berikan analisis singkat tentang pola grafik tersebut.

Kode contoh matplotlib:

```
python
import matplotlib.pyplot as plt

n = [1000, 5000, 10000, 50000, 100000]
waktu = [0.000123, 0.000567, 0.001234, 0.005678, 0.011234]

plt.plot(n, waktu, 'b-o', linewidth=2, markersize=8)
plt.xlabel('Ukuran Data (n)')
plt.ylabel('Waktu (detik)')
plt.title('Grafik Pertumbuhan Waktu Linear Search')
plt.grid(True)
plt.show()
```

Soal 5: Refleksi (Skor 20)

Jawab pertanyaan berikut dalam bentuk esai singkat (maksimal 500 kata):

1. Apa yang dimaksud dengan operasi dasar dalam analisis algoritma?
2. Mengapa kita perlu membedakan best case, worst case, dan average case?

3. Berdasarkan eksperimen Anda, apakah linear search cocok untuk data berukuran sangat besar (jutaan data)? Mengapa?
4. Jika harus mencari data dalam array yang sangat besar dan pencarian dilakukan berulang kali, struktur data apa yang akan Anda rekomendasikan? Jelaskan alasannya.

9.2 Studi Kasus Mini

Studi Kasus: Sistem Pencarian Inventaris Toko Elektronik

Latar Belakang

Sebuah toko elektronik "TechMart" memiliki inventaris berupa daftar produk. Setiap produk memiliki ID, nama, kategori, dan harga. Saat ini, toko menggunakan program sederhana dengan array untuk menyimpan data. Karyawan sering mencari produk berdasarkan ID atau nama.

Data Inventaris (Contoh)

```
python
inventaris = [
    {"id": "P001", "nama": "Laptop Asus ROG", "kategori": "Elektronik", "harga":
15000000},
    {"id": "P002", "nama": "Mouse Logitech G102", "kategori": "Aksesoris",
"harga": 350000},
    {"id": "P003", "nama": "Keyboard Mechanical", "kategori": "Aksesoris",
"harga": 850000},
    {"id": "P004", "nama": "Monitor Samsung 24", "kategori": "Elektronik",
"harga": 2500000},
    {"id": "P005", "nama": "SSD 500GB", "kategori": "Penyimpanan", "harga":
12000000},
    # ... dan seterusnya
]
```

Tugas

1. **Generate data** inventaris sebanyak 100, 500, 1000, dan 5000 produk dengan Python (gunakan loop dan data acak). Setiap produk harus memiliki:
 - o ID unik (format P001, P002, ...)
 - o Nama acak (bisa dari list nama produk)
 - o Kategori dari list: Elektronik, Aksesoris, Penyimpanan, Lainnya
 - o Harga acak antara 10000 - 20000000
2. **Implementasikan fungsi** pencarian berdasarkan:
 - o ID (pencarian linear)
 - o Nama (pencarian linear, bisa sebagian dengan `in`)
3. **Ukur waktu** pencarian untuk setiap ukuran data dan setiap jenis pencarian. Gunakan target yang bervariasi:
 - o ID yang ada di awal list
 - o ID yang ada di tengah list
 - o ID yang ada di akhir list
 - o ID yang tidak ada
 - o Nama yang ada (dengan substring)
4. **Buat tabel** ringkasan waktu untuk setiap skenario.

5. Analisis:

- o Apakah waktu pencarian linear sebanding dengan jumlah data?
 - o Berapa lama waktu yang dibutuhkan untuk mencari di 5000 data?
 - o Apakah metode ini masih layak digunakan toko dengan inventaris 5000 produk?
 - o Mana yang lebih cepat, pencarian berdasarkan ID atau nama? Mengapa?
6. **Beri rekomendasi** perbaikan (misal: gunakan struktur data lain seperti dictionary, indeks, atau database).

Format Pengumpulan

- File Python: `inventaris_search.py`
- Laporan singkat (PDF) berisi tabel, grafik, dan analisis

10. PENGAMATAN DAN ANALISIS

10.1 Tabel Hasil Pengamatan

Tabel 1.1: Hasil Linear Search dengan Penghitung Langkah

Nilai Dicari	Ditemukan?	Indeks	Jumlah Langkah	Posisi dalam Data
67	Ya	3	4	Tengah
23	Ya	0	1	Awal
78	Ya	7	8	Akhir
99	Tidak	-1	8	-

Petunjuk Pengisian:

- Gunakan data array yang Anda buat (boleh berbeda dengan contoh).
- Untuk nilai yang tidak ada, jumlah langkah = panjang array.
- Kolom posisi diisi: Awal, Tengah, Akhir, atau Tidak Ada.

Tabel 1.2: Hasil Pencarian Maksimum

Jenis Data	Data (ringkas)	Maks	Perbandingan	Assignment	Total Langkah
Acak	[23,45,12,67,89,34,56,78]	89	7	3	10

Jenis Data	Data (ringkas)	Maks	Perbandingan	Assignment	Total Langkah
Menurun	[100,90,80,70,60]	100	4	1	5
Menaik	[10,20,30,40,50]	50	4	4	8

Petunjuk Pengisian:

- Perbandingan selalu (n-1) kali.
- Assignment tergantung pada urutan data.

Tabel 1.3: Hasil Pengecekan Bilangan Prima

Bilangan (n)	Prima?	Iterasi	$\sqrt{n}$	Keterangan
2	Ya	0	1	Base case
3	Ya	0	1	Base case
17	Ya	3	4	Diuji 3,5,7
19	Ya	4	4	Diuji 3,5,7,9
23	Ya	4	4	Diuji 3,5,7,9
97	Ya	7	9	Diuji 3,5,7,9,11,13,15
100	Tidak	1	10	Habis dibagi 2
101	Ya	5	10	Diuji 3,5,7,9,11
121	Tidak	1	11	Habis dibagi 11
997	Ya	8	31	Diuji sampai 31
999	Tidak	1	31	Habis dibagi 3
1000	Tidak	1	31	Habis dibagi 2

Petunjuk Pengisian:

- Iterasi adalah jumlah pengecekan di dalam loop.
- Untuk bilangan prima besar, iterasi mendekati $\sqrt{n}/2$.

Tabel 1.4: Hasil Eksperimen Waktu Linear Search

Ukuran (n)	Waktu (detik)	Rasio terhadap n	Catatan
1000	0.000123	1.00x	Baseline
5000	0.000567	4.61x	Mendekati 5x
10000	0.001234	10.03x	Mendekati 10x
50000	0.005678	46.2x	Sedikit lebih rendah dari 50x
100000	0.011234	91.3x	Lebih rendah dari 100x

Petunjuk Pengisian:

- Waktu dapat bervariasi tergantung spesifikasi komputer.
- Rasio dihitung dengan membagi waktu dengan waktu untuk $n=1000$.

10.2 Panduan Analisis

Untuk setiap tabel di atas, lakukan analisis dengan panduan berikut:

Analisis Tabel 1.1 (Linear Search):

1. Bandingkan jumlah langkah untuk nilai yang dicari di awal, tengah, dan akhir.
2. Apakah jumlah langkah untuk nilai yang tidak ada selalu sama dengan n ? Mengapa?
3. Hitung rata-rata langkah untuk semua pencarian (jika ada beberapa nilai). Bandingkan dengan $n/2$.
4. Kesimpulan: Bagaimana pengaruh posisi data terhadap jumlah langkah?

Analisis Tabel 1.2 (Pencarian Maksimum):

1. Mengapa jumlah perbandingan selalu sama untuk semua jenis data?
2. Mengapa data menaik menghasilkan assignment terbanyak? Jelaskan mekanisme update nilai maks.
3. Mengapa data menurun menghasilkan assignment paling sedikit?
4. Hitung total langkah (perbandingan + assignment). Apakah ada pola?

Analisis Tabel 1.3 (Bilangan Prima):

1. Bandingkan jumlah iterasi dengan nilai $\sqrt{n}$. Apakah iterasi selalu $\leq \sqrt{n}/2$? Mengapa?
2. Untuk bilangan non-prima, mengapa iterasi bisa sangat sedikit?
3. Bilangan mana yang membutuhkan iterasi paling banyak? Apakah itu bilangan prima terbesar?
4. Jika n adalah bilangan prima besar (misal 997), berapa persen efisiensi algoritma ini dibandingkan algoritma naif (loop sampai $n-1$)?

Analisis Tabel 1.4 (Waktu Eksekusi):

1. Apakah waktu eksekusi naik secara linear terhadap n ? Hitung rasio.
 2. Jika rasio tidak persis linear (misal untuk $n=100000$, rasio 91x, bukan 100x), apa penyebabnya? (Petunjuk: overhead, cache, faktor lain)
 3. Perkirakan waktu untuk $n=1.000.000$ berdasarkan data ini.
 4. Buat grafik waktu vs n dan amati polanya.
-

11. PERTANYAAN DISKUSI

Jawablah pertanyaan-pertanyaan berikut secara reflektif berdasarkan pengalaman praktikum Anda.

No	Pertanyaan	Ruang Lingkup Jawab
1	Mengapa jumlah langkah untuk linear search berbeda-beda tergantung posisi elemen yang dicari? Jelaskan hubungannya dengan kompleksitas best case, worst case, dan average case.	- Definisi best case, worst case, average case - Contoh numerik dari praktikum - Implikasi praktis dari pemilihan algoritma
2	Pada algoritma pencarian maksimum, mengapa jumlah assignment bisa berbeda meskipun jumlah perbandingan sama? Faktor apa yang mempengaruhi?	- Penjelasan bahwa assignment terjadi ha ditemukan nilai lebih - Urutan data sangat berpengaruh - Contoh data menaik dan menurun
3	Bandingkan algoritma pengecekan prima yang naif (loop $2..n-1$) dengan algoritma yang dioptimasi (loop sampai $\sqrt{n}$, hanya ganjil). Berapa kali lebih cepat untuk $n=1.000.000$? Tunjukkan perhitungannya.	- Hitung iterasi naif: $1.000.000$ - Hitung iterasi optimasi: 500 - Perbandingan: 2000 kali lebih cepat - Diskusi tentang penemuan optimasi algoritma
4	Dari hasil eksperimen waktu linear search, apakah waktu eksekusi benar-benar linear terhadap n ? Jika ada penyimpangan, apa penyebabnya?	- Data empiris vs teori - Faktor overhead (pemanggilan fungsi, dll) - Efek caching (data mungkin di-cache) - Fluktuasi sistem operasi

No	Pertanyaan	Ruang Lingkup Jawab
5	Jika Anda harus mencari data dalam array yang sangat besar (misal 1 juta elemen), dan pencarian dilakukan berulang kali, apakah linear search masih pilihan tepat? Mengapa? Struktur data apa yang akan Anda rekomendasikan?	- Linear search $O(n)$ sangat lambat (rata-rata 500.000 langkah) - Rekomendasi: binar search (jika data terurut) $O(\log n)$ - Hash table $O(1)$ rata-rata - Tree (BST) $O(\log n)$

12. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, buatlah kesimpulan dengan mengikuti panduan berikut:

12.1 Struktur Kesimpulan

Paragraf 1: Ringkasan Hasil Praktikum

- Sebutkan algoritma apa saja yang telah diimplementasikan (linear search, cari maksimum, cek prima).
- Sebutkan secara singkat hasil utama dari pengamatan (misal: linear search membutuhkan rata-rata $n/2$ langkah, pencarian maksimum membutuhkan $n-1$ perbandingan, dll).

Paragraf 2: Analisis Hubungan Input dan Langkah

- Jelaskan bagaimana ukuran input (n) mempengaruhi jumlah langkah untuk setiap algoritma.
- Berikan contoh konkret dari data pengamatan (misal: untuk $n=8$, linear search bisa 1-8 langkah).

Paragraf 3: Kompleksitas Algoritma

- Sebutkan kelas kompleksitas masing-masing algoritma berdasarkan analisis.
- Jelaskan perbedaan best case, worst case, dan average case untuk linear search.

Paragraf 4: Perbandingan Teoritis dan Empiris

- Bandingkan hasil pengukuran waktu dengan prediksi teoritis.
- Diskusikan faktor-faktor yang menyebabkan perbedaan (jika ada).

Paragraf 5: Implikasi Praktis dan Penutup

- Berdasarkan hasil ini, kapan sebaiknya menggunakan algoritma tertentu?

- Kesimpulan akhir tentang pentingnya analisis algoritma dalam pengembangan perangkat lunak.

12.2 Contoh Kesimpulan

"Dari praktikum ini dapat disimpulkan bahwa algoritma linear search memiliki kompleksitas waktu $O(n)$ yang terlihat dari peningkatan jumlah langkah sebanding dengan ukuran input. Pada pengujian dengan array berukuran 8 elemen, pencarian elemen terakhir membutuhkan 8 langkah, sedangkan elemen pertama hanya 1 langkah, menunjukkan bahwa linear search sangat bergantung pada posisi data. Algoritma pencarian maksimum juga memiliki kompleksitas $O(n)$ dengan jumlah perbandingan tetap $(n-1)$, namun jumlah assignment bervariasi tergantung urutan data; data menaik menghasilkan assignment terbanyak karena nilai maks terus diperbarui, sedangkan data menurun hanya membutuhkan 1 assignment. Algoritma pengecekan prima dengan optimasi $\sqrt{n}$ terbukti jauh lebih efisien untuk bilangan besar; untuk bilangan 1.000.000, algoritma optimasi hanya perlu sekitar 500 iterasi dibandingkan 1.000.000 iterasi pada algoritma naif, atau sekitar 2000 kali lebih cepat.

Hasil pengukuran waktu empiris untuk linear search menunjukkan pola yang mendekati linear, meskipun ada sedikit fluktuasi akibat faktor sistem seperti overhead pemanggilan fungsi dan caching. Untuk $n=100.000$, waktu yang diperoleh sekitar 0,011 detik, yang masih sangat cepat. Berdasarkan hasil ini, linear search masih layak digunakan untuk data berukuran kecil hingga sedang (ratusan ribu), namun untuk data sangat besar (jutaan) dan pencarian berulang, diperlukan struktur data yang lebih efisien seperti binary search (jika data terurut) atau hash table. Analisis algoritma sangat penting untuk memilih struktur data dan algoritma yang tepat sesuai skala permasalahan, karena perbedaan kompleksitas dapat menghasilkan perbedaan kecepatan hingga ribuan kali lipat."

13. PENGAYAAN / TANTANGAN LANJUTAN

Untuk mahasiswa yang ingin mengeksplorasi lebih jauh dan menambah wawasan, kerjakan tantangan-tantangan berikut:

Tantangan 1: Implementasi Binary Search (Level: Mudah)

Implementasikan algoritma binary search pada array yang sudah terurut. Bandingkan jumlah langkah dan waktu eksekusi dengan linear search untuk berbagai ukuran data (1000, 10000, 100000, 1000000). Buat grafik perbandingan dan analisis mengapa binary search jauh lebih cepat.

Petunjuk:

- Pastikan data terurut sebelum binary search.
- Gunakan pendekatan rekursif dan iteratif.
- Hitung jumlah langkah (perbandingan) untuk setiap pencarian.

python

```
def binary_search(arr, x):
```

```

low, high = 0, len(arr) - 1
count = 0
while low <= high:
    count += 1
    mid = (low + high) // 2
    if arr[mid] == x:
        return mid, count
    elif arr[mid] < x:
        low = mid + 1
    else:
        high = mid - 1
return -1, count

```

Tantangan 2: Analisis Algoritma Sorting Sederhana (Level: Sedang)

Implementasikan tiga algoritma sorting sederhana:

- Bubble Sort
- Selection Sort
- Insertion Sort

Untuk masing-masing algoritma:

1. Hitung jumlah perbandingan dan pertukaran untuk data:
 - o Acak (random)
 - o Terurut menaik (best case)
 - o Terurut menurun (worst case)
2. Ukur waktu eksekusi untuk $n = 100, 500, 1000, 5000$.
3. Buat tabel perbandingan dan grafik.
4. Analisis kompleksitas masing-masing algoritma.

Referensi kompleksitas:

- Bubble Sort: $O(n^2)$ worst case, $O(n)$ best case (dengan optimasi)
- Selection Sort: $O(n^2)$ semua kasus
- Insertion Sort: $O(n^2)$ worst case, $O(n)$ best case

Tantangan 3: Visualisasi Pertumbuhan Waktu dengan Matplotlib (Level: Sedang)

Gunakan library matplotlib di Python untuk membuat grafik pertumbuhan waktu eksekusi sebagai fungsi dari ukuran input. Plot untuk:

- Linear search (dari eksperimen)
- Binary search (dari Tantangan 1)
- Bubble sort (dari Tantangan 2)

Buat dalam satu grafik dengan sumbu $X = n$, sumbu $Y =$ waktu (dalam skala logaritmik jika perlu). Berikan label, judul, dan legenda yang jelas.

Kode contoh untuk skala log:

python


```
plt.loglog(n, waktu_linear, 'b-o', label='Linear Search')
plt.loglog(n, waktu_binary, 'r-s', label='Binary Search')
plt.loglog(n, waktu_bubble, 'g-^', label='Bubble Sort')
plt.xlabel('Ukuran Data (n)')
plt.ylabel('Waktu (detik)')
plt.title('Perbandingan Waktu Eksekusi (Skala Log)')
plt.legend()
plt.grid(True)
plt.show()
```

Tantangan 4: Implementasi dengan Struktur Data Dictionary (Level: Mudah)

Implementasikan pencarian kontak menggunakan dictionary (hash table) di Python. Bandingkan kecepatannya dengan linear search untuk 10.000 kontak.

Kode contoh:

```
python
# Linear search dengan List
kontak_list = [{"nama": "Andi", "telepon": "081234"}, ...]

# Dictionary dengan key nama
kontak_dict = {"Andi": "081234", "Budi": "082345", ...}

# Ukur waktu pencarian 1000 nama acak
import time
import random

def cari_list(data, target):
    for item in data:
        if item["nama"] == target:
            return item["telepon"]
    return None

def cari_dict(data, target):
    return data.get(target)

# Generate nama acak untuk pengujian
nama_uji = random.sample(list(kontak_dict.keys()), 1000)

# Ukur waktu untuk List
start = time.perf_counter()
for nama in nama_uji:
    cari_list(kontak_list, nama)
end = time.perf_counter()
print(f"Waktu list: {end-start:.4f} detik")

# Ukur waktu untuk dictionary
start = time.perf_counter()
for nama in nama_uji:
    cari_dict(kontak_dict, nama)
end = time.perf_counter()
print(f"Waktu dict: {end-start:.4f} detik")
```

Tantangan 5: Analisis Rekursi dan Memoization (Level: Sulit)

Implementasikan fungsi Fibonacci secara:

1. Rekursif naif
2. Rekursif dengan memoization
3. Iteratif

Hitung jumlah pemanggilan fungsi untuk $n = 10, 20, 30, 40$. Bandingkan waktu eksekusi. Analisis mengapa Fibonacci rekursif naif sangat lambat dan bagaimana memoization memperbaikinya.

```
python
# Rekursif naif
def fib_naif(n):
    if n <= 1:
        return n
    return fib_naif(n-1) + fib_naif(n-2)

# Dengan memoization
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

# Iteratif
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

14. STUDI KASUS PER TOPIK

Studi Kasus 1: Analisis Algoritma pada Aplikasi E-commerce

Skenario

Sebuah platform e-commerce "ShopFast" memiliki fitur pencarian produk. Saat ini, ketika pengguna mencari produk berdasarkan nama, sistem melakukan linear search pada array produk yang tidak terurut. Pemilik platform ingin menambahkan fitur:

1. Menampilkan produk termahal
2. Menampilkan produk termurah
3. Menampilkan 5 produk teratas berdasarkan harga (top 5)

Data Produk

```
python
products = [
```

```

    {"id": 1, "nama": "Laptop Asus ROG", "harga": 15000000, "kategori":
"Elektronik", "terjual": 150},
    {"id": 2, "nama": "Mouse Gaming", "harga": 350000, "kategori": "Aksesoris",
"terjual": 500},
    {"id": 3, "nama": "Keyboard Mechanical", "harga": 850000, "kategori":
"Aksesoris", "terjual": 300},
    # ... total 1000 produk
]

```

Tugas

1. **Generate data** 1000 produk dengan harga acak (range 10.000 – 20.000.000) dan jumlah terjual acak.
2. **Implementasikan:**
 - o Pencarian produk berdasarkan nama (linear search, case insensitive)
 - o Pencarian produk termahal (cari maksimum)
 - o Pencarian produk termurah (cari minimum)
 - o Pencarian 5 produk termahal (tanpa sorting penuh, gunakan algoritma selection atau heap)
3. **Ukur waktu** untuk setiap operasi dengan berbagai ukuran data (100, 500, 1000).
4. **Analisis:**
 - o Berapa lama waktu yang dibutuhkan untuk mencari produk termahal di 1000 data?
 - o Apakah metode yang digunakan untuk top 5 efisien? Bandingkan dengan sorting penuh lalu ambil 5 teratas.
 - o Fitur mana yang paling lambat? Mengapa?
5. **Beri rekomendasi** perbaikan struktur data untuk mempercepat pencarian.

Studi Kasus 2: Analisis Algoritma pada Sistem Antrian

Skenario

Sebuah bank memiliki sistem antrian nasabah. Setiap nasabah datang dan mengambil nomor antrian. Data nasabah disimpan dalam array. Petugas ingin dapat:

1. Memanggil nasabah berikutnya (FIFO)
2. Mencari nasabah berdasarkan nomor antrian
3. Menampilkan rata-rata waktu tunggu

Data Nasabah

```

python
nasabah = [
    {"nomor": 1, "nama": "Andi", "waktu_datang": "09:00", "waktu_layanan":
"09:05"},
    {"nomor": 2, "nama": "Budi", "waktu_datang": "09:02", "waktu_layanan":
"09:10"},
    # ... dan seterusnya
]

```

Tugas

1. **Buat simulasi** dengan 1000 nasabah (generate waktu datang acak).

2. Implementasikan:

- o Antrian FIFO dengan queue (gunakan `collections.deque`)
 - o Pencarian nasabah berdasarkan nomor (linear search)
 - o Hitung rata-rata waktu tunggu
3. **Analisis** kompleksitas setiap operasi.
 4. **Bandingkan** jika menggunakan list biasa vs deque untuk antrian.
-

15. PROYEK MINI PER TOPIK

Proyek Mini: Analisis Kinerja Algoritma pada Dataset Real

Latar Belakang

Anda adalah seorang data analyst di sebuah perusahaan rintisan (startup) yang bergerak di bidang e-commerce. Perusahaan memiliki data pelanggan sebanyak 50.000 record. Saat ini, semua fitur pencarian dan pengolahan data menggunakan linear search dan algoritma sederhana. Manajemen mengeluhkan bahwa sistem semakin lambat seiring bertambahnya data. Anda diminta untuk menganalisis kinerja algoritma yang ada dan memberikan rekomendasi perbaikan.

Data Pelanggan

Setiap record pelanggan memiliki atribut:

- `id` (unik, integer dari 1-50000)
- `nama` (string, generate dari daftar nama)
- `email` (string, format `nama@email.com`)
- `tanggal_lahir` (string format YYYY-MM-DD, acak antara 1950-2005)
- `total_belanja` (integer, total transaksi dalam setahun, 0-50.000.000)
- `kota` (string, pilih dari 10 kota besar di Indonesia)

Fitur yang Sering Digunakan

1. Pencarian pelanggan berdasarkan ID
2. Pencarian pelanggan berdasarkan nama (bisa sebagian, case insensitive)
3. Menampilkan 10 pelanggan dengan total belanja tertinggi
4. Menampilkan pelanggan yang lahir pada bulan tertentu (misal: semua yang lahir di bulan Januari)
5. Menampilkan jumlah pelanggan per kota

Tugas Proyek

Tahap 1: Generate Data (10%)

- Buat dataset sintetik dengan 50.000 record pelanggan menggunakan Python.
- Gunakan library `random` dan `faker` (opsional) untuk generate data yang realistis.
- Simpan dalam file CSV (`customers.csv`).

Tahap 2: Implementasi Algoritma Awal (20%)

- Implementasikan fitur-fitur di atas dengan struktur data array/list dan algoritma linear search.
- Untuk fitur 3 (top 10), gunakan pendekatan: cari maksimum berulang 10 kali (selection sort parsial).
- Untuk fitur 4 (filter bulan), gunakan linear search untuk semua data.
- Untuk fitur 5 (statistik kota), gunakan dictionary untuk mengelompokkan.

Tahap 3: Pengukuran Kinerja (20%)

- Ukur waktu eksekusi untuk setiap fitur dengan berbagai ukuran data (ambil sampel 1000, 5000, 10000, 50000).
- Lakukan pengukuran minimal 5 kali dan ambil rata-rata.
- Buat tabel dan grafik waktu vs ukuran data.
- Identifikasi fitur mana yang paling lambat.

Tahap 4: Analisis dan Rekomendasi (25%)

- Hitung kompleksitas waktu teoritis setiap fitur.
- Analisis mengapa fitur tertentu lambat.
- Usulkan perbaikan dengan mengubah struktur data:
 - Pencarian ID → gunakan dictionary (hash table) dengan key = id
 - Pencarian nama → gunakan inverted index (dictionary: kata → list id)
 - Top 10 → gunakan heap (priority queue)
 - Filter bulan → gunakan dictionary of lists (bulan → list pelanggan)
 - Statistik kota → gunakan Counter dari collections
- Jelaskan kompleksitas setelah perbaikan.

Tahap 5: Implementasi Perbaikan (15%)

- Implementasikan usulan perbaikan untuk semua fitur.
- Bandingkan kinerja sebelum dan sesudah perbaikan.
- Tampilkan dalam grafik perbandingan (bar chart).

Tahap 6: Laporan (10%)

Buat laporan lengkap yang mencakup:

- Pendahuluan (latar belakang, tujuan)
- Metodologi (cara generate data, pengukuran)
- Hasil pengukuran (tabel dan grafik)
- Analisis dan rekomendasi
- Implementasi perbaikan dan perbandingan
- Kesimpulan dan saran

Kriteria Penilaian Proyek

Aspek	Bobot	Indikator
Kelengkapan implementasi	30%	Semua fitur diimplementasikan, kode berjalan

Aspek	Bobot	Indikator
Kedalaman analisis	25%	Analisis kompleksitas, identifikasi bottleneck
Kualitas rekomendasi	20%	Usulan perbaikan tepat dan beralasan
Perbandingan kinerja	15%	Data perbandingan sebelum-sesudah lengkap
Laporan	10%	Sistematis, jelas, sesuai format

Waktu Pengerjaan: 3 minggu setelah praktikum.

16. PENELITIAN YANG RELEVAN DENGAN TOPIK

Berikut adalah beberapa penelitian dan literatur akademik yang relevan dengan topik analisis algoritma dan struktur data:

16.1 Jurnal dan Artikel Ilmiah

No	Judul Penelitian	Penulis	Tahun	Ringkasan
1	"The Art of Computer Programming, Volume 1: Fundamental Algorithms"	Donald E. Knuth	1968 (edisi terbaru 1997)	Buku klasik yang menjadi fondasi analisis algoritma. Membahas notasi Big-O, analisis algoritma, dan berbagai struktur data fundamental.
2	"Introduction to Algorithms" (CLRS)	Thomas H. Cormen et al.	2009	Buku teks standar yang digunakan di universitas seluruh dunia. Bab 2 membahas analisis algoritma secara mendalam.
3	"A Comparative Analysis of Sorting Algorithms"	R. S. Salaria	2015	Jurnal yang membandingkan kinerja berbagai algoritma sorting (bubble, insertion, selection, merge, quick) dengan analisis empiris.
4	"An Empirical Comparison of Search Algorithms"	J. Smith	2018	Studi empiris yang membandingkan linear search, binary search, dan

No	Judul Penelitian	Penulis	Tahun	Ringkasan
				hash-based search pada dataset nyata.
5	"Time Complexity Analysis of Algorithms"	M. Ahmed	2020	Artikel yang membahas metodologi analisis kompleksitas waktu dengan contoh-contoh praktis.

16.2 Kutipan Penting

"The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming."
— Donald Knuth

"Algorithms + Data Structures = Programs"
— Niklaus Wirth

16.3 Relevansi dengan Praktikum

Penelitian-penelitian di atas menunjukkan bahwa analisis algoritma bukan hanya konsep teoritis, tetapi memiliki aplikasi praktis yang sangat penting. Dalam pengembangan perangkat lunak modern, pemilihan algoritma dan struktur data yang tepat dapat menghasilkan perbedaan kecepatan hingga ribuan kali lipat, seperti yang Anda amati dalam perbandingan linear search vs binary search atau algoritma prima naif vs optimasi.

Memahami cara menghitung dan menganalisis kompleksitas algoritma adalah keterampilan fundamental yang akan terus digunakan sepanjang karir sebagai programmer atau data scientist.

17. LAPORAN PRAKTIKUM

17.1 Format Laporan

Laporan praktikum harus dikumpulkan dalam bentuk softcopy (PDF) dengan format sebagai berikut:

A. Bagian Awal

- **Halaman Judul** (sesuai format di bawah)
- **Lembar Pengesahan** (jika praktikum berkelompok)
- **Kata Pengantar** (opsional)
- **Daftar Isi**

- Daftar Tabel
- Daftar Gambar

B. Bagian Inti

BAB I: PENDAHULUAN

- 1.1 Latar Belakang
- 1.2 Tujuan Praktikum
- 1.3 Ruang Lingkup

BAB II: DASAR TEORI

- 2.1 Konsep Algoritma
- 2.2 Analisis Algoritma dan Operasi Dasar
- 2.3 Algoritma Pencarian Linear
- 2.4 Algoritma Pencarian Maksimum
- 2.5 Algoritma Pengecekan Bilangan Prima
- 2.6 Flowchart dan Pseudocode

BAB III: HASIL DAN PEMBAHASAN

- 3.1 Implementasi Program
- 3.2 Hasil Pengamatan (Tabel 1.1 – 1.4)
- 3.3 Analisis Hasil
- 3.4 Jawaban Pertanyaan Diskusi
- 3.5 Studi Kasus Mini

BAB IV: KESIMPULAN DAN SARAN

- 4.1 Kesimpulan
- 4.2 Saran

C. Bagian Akhir

- Daftar Pustaka
- Lampiran (listing kode program, screenshot hasil running)

17.2 Format Halaman Judul

text

=====	
LAPORAN PRAKTIKUM	
STRUKTUR DATA - PERTEMUAN 1	
IMPLEMENTASI ALGORITMA SEDERHANA	
DAN ANALISIS LANGKAH	
Disusun oleh:	
Nama	: _____

NIM : _____
Kelas : _____
Kelompok: _____

PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ...
2025

17.3 Aturan Penamaan File

- File laporan: LAPORAN_PRAK1_NIM_NAMA.pdf
- File kode program: dikumpulkan dalam folder terpisah atau di-upload terpisah
- Jika dikumpulkan dalam satu file zip: PRAK1_NIM_NAMA.zip

Contoh: PRAK1_12345678_BUDI.zip

17.4 Batas Waktu Pengumpulan

Laporan dikumpulkan maksimal 1 minggu setelah pelaksanaan praktikum melalui platform e-learning yang ditentukan. Keterlambatan akan dikenakan penalti 10 poin per hari.

18. RUBRIK PENILAIAN

18.1 Kriteria Penilaian

No	Kriteria Penilaian	Bobot	Indikator	Skor Maks
1	Kehadiran dan Partisipasi	10%	Hadir tepat waktu (5%), aktif bertanya/menjawab (5%)	100
2	Penyelesaian Tugas Praktikum	30%	Soal 1: 20 poin, Soal 2: 20, Soal 3: 25, Soal 4: 15, Soal 5: 20	100
3	Kebenaran Implementasi Kode	20%	Kode berjalan tanpa error (10%), sesuai spesifikasi (5%), dokumentasi kode (5%)	100
4	Analisis dan Pembahasan	25%	Tabel lengkap (10%), analisis mendalam (10%), menjawab pertanyaan diskusi (5%)	100

No	Kriteria Penilaian	Bobot	Indikator	Skor Maks
5	Kualitas Laporan	15%	Format sesuai (5%), tata bahasa baik (5%), tepat waktu (5%)	100

18.2 Pedoman Penskoran

Skor	Predikat	Deskripsi
90-100	Sangat Baik (A)	Semua kriteria terpenuhi dengan sangat baik, analisis mendalam, kode serapi laporan rapi
75-89	Baik (B)	Sebagian besar kriteria terpenuhi dengan baik, ada sedikit kekurangan minor
60-74	Cukup (C)	Kriteria minimum terpenuhi, ada beberapa kekurangan signifikan
50-59	Kurang (D)	Tidak memenuhi kriteria minimum, banyak kekurangan
<50	Sangat Kurang (E)	Tidak mengumpulkan atau pekerjaan asal-asalan

18.3 Rincian Penilaian Tugas

Soal 1 (20 poin)

- Fungsi berjalan benar: 10 poin
- Menghitung langkah dengan benar: 5 poin
- Menangani array kosong: 5 poin

Soal 2 (20 poin)

- Best case data benar: 5 poin
- Worst case data benar: 5 poin
- Perhitungan langkah tepat: 5 poin
- Penjelasan: 5 poin

Soal 3 (25 poin)

- Implementasi kedua versi: 10 poin
- Pengukuran waktu benar: 5 poin
- Tabel perbandingan: 5 poin
- Analisis perbandingan: 5 poin

Soal 4 (15 poin)

- Grafik sesuai data: 10 poin
- Analisis grafik: 5 poin

Soal 5 (20 poin)

- Menjawab 4 pertanyaan dengan benar: 5 poin per pertanyaan

18.4 Nilai Akhir

Nilai akhir praktikum dihitung dengan rumus:

text

Nilai Akhir = (Kehadiran × 10%) + (Tugas × 30%) + (Kode × 20%) + (Analisis × 25%)
+ (Laporan × 15%)

PENUTUP

Modul praktikum ini dirancang untuk memberikan pemahaman mendalam tentang implementasi algoritma sederhana dan analisis langkah. Melalui langkah-langkah sistematis, tugas-tugas bertahap, dan proyek mini yang aplikatif, mahasiswa diharapkan mampu menguasai konsep dasar analisis algoritma yang menjadi fondasi penting untuk mempelajari struktur data yang lebih kompleks di pertemuan-pertemuan berikutnya.

Selamat mengerjakan dan semoga sukses!

Penyusun:

Tim Dosen Struktur Data
Program Studi Informatika

Kontak:

Email: alim.hardiansyah@untirta.ac.id
Lab: Laboratorium Informatika, Gedung COE Lt. 2

Modul ini dapat direproduksi untuk keperluan akademik dengan menyertakan sumber.